NEW STUDENT SEMINAR · 项目答辩

昨日圆车

HEX WAGON

六边形网格 · 双人回合制战棋

规则引擎 / 启发式 AI / WebSocket 联机 / 自建兵种工坊



第 XX 组 · 作者 1 作者 2 作者 3 作者 4

2026 年 11 月

02 · 一页看懂

一页看懂这个项目

7 种兵种

含光环、召唤与直线攻击

4 种模式

本地 / 人机 / 联机 / 教学

819 项断言

规则 / AI / 联机 三套自动化

1 条命令部署

静态资源 + Node 服务端

浏览器打开就能玩：一份规则代码，客户端与服务端共用

单机、AI、联机三条路径走的是同一个 engine.js，所以「客户端能下、服务端不认」这类分歧在结构上不存在

- 纯静态前端，无构建步骤
- 规则层零依赖 DOM，命令行可测
- 玩家可自建兵种并上传

今天要讲：为什么做（背景）→ 别人做到哪（现状）→ 我们怎么做（方案）→ 做出来什么样（展示）→ 还能怎么走（展望）

03 · 应用背景

为什么选一款战棋来做

不是因为它好写，而是因为它把课程里的四类知识压在同一张桌子上

01 规则可形式化

整局只用「棋盘 + 棋子 + 费用」表示，
每条规则都能写成一条断言：

能不能放、打几点、谁先死、
断联之后连锁掉谁。

→ 可以做成自动化测试

02 状态必须一致

一击致死要在同一时刻判定，
断联移除还会引发连锁。

算错一步，玩家当场就能看见；
联机时两边还会各算一份。

→ 必须做「单份规则 + 服务端权威」

03 需求会一直长

从 3 个兵种加到 7 个，
再加光环、替换、自建兵种。

如果每种新玩法都要改一遍
渲染和网络代码，就会失控。

→ 考验的是「可扩展」而不是「能跑」

换句话说：它同时压住了 数据结构（六边形）、算法（AI 与结算）、网络（联机同步）、软件设计（可扩展）四条线。

这也是我们最终没有做「贪吃蛇 / 2048 类」小游戏的原因——它们很难长到需要架构的程度。

04 · 应用现状

同类产品做到了哪一步

类型 / 代表	强项	短板	与本项目的关系
单机策略战棋 火焰纹章、高级战争	规则深、关卡厚、 兵种与地形互相咬合	体量大、周期长， 不做实时联机	借它的战斗模型， 缩到 15 分钟一局
网页回合制对战 在线棋类、卡牌对战	开房间即玩， 联机与断线重连成熟	玩法多为既成规则， 玩家只能换牌组	借它的房间模型， 但规则要能自己长
创作型平台 工坊、关卡编辑器	UGC 生态强， 创作者能持续产出	规则常靠脚本引擎承载， 数值与平衡难管	借它的思路， 但用「白名单字段」约束

定位空档：**小体量（一局十几分钟） + 玩家可自建兵种 + 真联机**
三者同时具备的少，因为前两条各自都会把后两条的成本推高。

05 · 技术现状

四条技术线，我们各自站在哪

六边形网格

轴向坐标 + 立方坐标约束

主流做法是 Red Blob 的 axial / cube 体系：
距离、邻居、像素映射都有闭式公式。

```
d = (|dq| + |dr| + |dq+dr|) / 2
```

棋类 AI

从 Minimax 到 MCTS 到 AlphaZero

AlphaZero 已证明「自对弈 + 价值网络」可行，
但训练成本与本项目规模不匹配。

我们的选择：一步前瞻 + 手写评估函数

实时通信

WebSocket 是网页双向通信的标准答案

HTTP 轮询延迟高、开销大；SSE 只能单向下发。
对局是「双方随时可能动作」，长连接更合适。

我们的选择：ws 库 + 心跳 + 座位令牌重连

内容扩展

开放内容通常走「数据驱动 + 白名单」

让玩家提交数据而非代码：字段固定、数值夹紧，
服务端二次校验，避免自定义内容变成攻击面。

我们的选择：REST 上传 + 对局只传编号

四层结构，规则只有一份

调用方向

表现层
画布渲染 · 面板 · 输入
`renderer.js` `main.js` `tutorial.js`

应用层
对局控制 · 联机 · 设置
`main.js` `net.js` `settings.js` `mods.js`

规则层
唯一真相来源
`engine.js` (浏览器 & Node 同一文件)

传输层
协议 · 存储
`WebSocket /ws` `REST /api/mods` 静态文件

同一份 engine.js

① 浏览器单机 / AI
直接 new Game()

② AI 自对弈脚本
node 里跑同一套规则

③ 服务端校验
require 同一个文件

所以联机不存在
「两套规则不一致」

可测
规则层不碰 DOM，命令行跑断言

同构
UMD 双模式，浏览器与 Node 通用

可扩展
兵种是数据，不是分支代码

规则引擎：一回合只走四步

回合流程（可反复执行）

1. **放置**
   扣费 · 校验连通 · 可选择方向
2. **替换**
   点己方棋子选中，再点兵种卡换掉
3. **结算**
   累计伤害 → 同时判定 → 移除
4. **结束回合**
   +3 费 · 换手 · 断联检查

放置合法性 = 连通块边缘

从己方大本营做一次 BFS，能走到且与己方棋子相邻的空格才可放。

被隔断的棋子每回合检查一次，断开就整串消失（可连锁）。

```
neighbors → BFS → edge(owner)
```

结算三阶段，同一份快照

① 先算全场伤害表（此时谁都不死）② 再同时判定 hp ≤ dmg

③ 最后一次性移除。这样「互击同归于尽」不会因为遍历顺序变成「先打的人赢」。

光环是数据，同类取最高

半径 + 效果 + 数值三个字段；同种效果叠加会破坏平衡，故只取最大值。

AI：评估函数比搜索深度更要紧

评估函数由五项组成（数值经自对弈调试）

棋子价值 按攻击力 / 血量 / 射程合成，光环按边际贡献折算

残血折价 血量越低越便宜，被打一次再折一次

吃子估值 先算伤害表，本回合会死的棋子按已死计分

推进施压 离对方大本营越近越好，能一击打爆直接返回胜势

子力数量 每个存活棋子给一点「场面分」

为什么不套 MCTS

单回合可行动作数在百级，分支因子大但深度浅；纯搜索在 10 ms 预算里只能看两三步，收益不如把评估写准——实测加吃子项后对旧版 7 胜 2 负。

决策方式：枚举 + 一步前瞻

枚举所有「花钱方式」

买哪一类 × 放哪个格 × 挂哪个方向 × 换哪个子

每个候选：克隆局面 → 落子 → 估分

`clone()` → `place()` → `evaluate()` 取最高分

单次选点约 10 ms；走完一整回合约 30 ms

效果（固定随机种子，各 20 局）

对随机走子		20 : 0
对上一版 AI		7 胜 · 2 负 · 11 平
单次选点耗时	≈ 10 ms	（自对弈 6 局，记录每步平均）

平局多是因为双方都退化成对耗——这本身说明评估函数偏保守

联机与自建兵种：都不信任客户端

① 联机：服务端权威

建房 / 加入
4 位房号 + 座位令牌
→
只发「意图」
place / resolve / end
→
服务端算一遍
同一个 engine.js
→
广播全量状态
含双方费用与手牌
→
客户端重绘
不做增量猜测

为什么发全量而不是增量？一局状态只有几 KB，重连、观战、悔棋都变成「换一个快照」；代价是带宽，可接受。

② 自建兵种：只传编号，定义由服务端提供

工坊编辑
血量 / 攻击 / 光环
→
POST /api/mods
8 KB 上限 + 配额
→
白名单字段 + 夹值
hp 1–30 · cost 0–30
→
对局只带 modIds
形如 mod_cb4385…
→
服务端取定义
注入本局 types

自定义 ≠ 可执行代码

玩家提交的是一组数值，不是脚本；
服务端重新校验并夹紧，越界就截断。
→ 不被上传内容拖进注入面

双方必须用同一份定义

若由客户端各自上传，房主改一次数值
就能让对手看到不同的兵——所以按 id 取。
→ 同 id 同定义，两个客户端才谈得上一致

额度与删除凭据

每个客户端免密最多 4 个自定义棋子；
删除需要上传时返回的 secret。
→ 防止把公共库当图床

10 · 效果展示

两个页面，从入座到开打

入口页 index.html



四种模式一屏选完

本地双人 / 人机 / 联机开房 / 新手教学，右侧是「继续对局」与在线房间数；设置与规则都在这一页，不来回跳。

对局页 game.html



一屏装下决策需要的全部信息

左：双方兵力与战报；中：棋盘与落点提示；右：选中棋子的详细数值；底：六张兵种卡与「结算 / 结束回合」。

三个细节，决定它好不好用



① 放之前就知道能打到谁

选好兵种后把鼠标停在合法格上，
金色虚线格 = 放这里能覆盖的范围，
范围内的敌人套红圈。



② 场上棋子可以直接换

点己方棋子 → 卡片显示它现在的
实际攻防（含光环）→ 再点任意兵种卡
即可替换，费用按新兵种结算。



③ 自建兵种工坊

名称 / 图标 / 血量 / 攻击 /
射程 / 费用 / 光环，右侧实时
预览卡片长相。

三处都有同一个设计取向：把「结果」提前画给玩家看，而不是让玩家下完再发现不对。

工坊里定义的兵种会出现在商店第 7 张卡之后；联机时由服务端按编号补给双方，所以两边看到的是同一个兵。

12 · 验证与上线

怎么证明它是对的

三套自动化测试，合计 819 项断言（< 3 秒跑完）

规则测试 678

放置 / 结算 / 断联 / 光环 / 替换 / 序列化

每条规则至少一个「反例断言」：不该允许的必须被拒绝

AI 测试 97

合法着法 / 不超预算 / 对随机 20:0 / 不出现空回合

联机测试 44

建房 / 越权 / 重连 / 求和 / 认输 / mod 注入

测试不是摆设：改完光环与替换功能后，测试先红后绿，才敢把改动推到线上。

已部署在公网（阿里云轻量应用服务器）

`http://47.117.128.251:3000`

Ubuntu 24.04 · 2 vCPU / 2 GB · 端口 3000

systemd 托管，崩溃自动重启；一条命令热更新

`GET /health → ok`　`GET /stats → {rooms, waiting, players}`

域名已注册（yesterday-wagon.xyz），ICP 备案在流程中

答辩前修掉的真实问题（都写了回归断言）

▪ 服务器上的自建棋子选不中	列表行点击未绑定 / 定义只查本地
▪ 删掉的兵种刷新后又回来了	删除只清本地，缓存又合并回来
▪ 狙击兵选方向时看不到箭头	删角标时误删了绘制调用

三个问题有个共同点：都是「看起来存下来了，其实没有」。

所以后来加了读回校验：写完立刻读一遍模型，而不是相信返回值。

这也是我们在测试里坚持「正例 + 反例」的原因。

13 · 总结与展望

做成了什么，还差什么

三条结论

规则只有一份，所以「一致」是结构保证

客户端、AI、服务端 require 同一个 engine.js；
联机争议从「谁算对了」变成「服务端说了算」。

兵种是数据，所以加内容不改代码

从 3 个兵种加到 7 个、再加玩家自建的兵种，
规则层只改了一张表；渲染与面板自动跟上。

把「不确定」交给测试，而不是交给手感

819 项断言 + 两组 AI 对抗数据，改数值或改规则
之后能立刻知道「是不是变差了」。

谢谢，欢迎提问

还差什么 / 下一步

数值平衡还没有系统方法

自对弈里坦克几乎没人买（6 局 0 次）——这是数据
说出来的问题，下一步要用胜率回归来调价。

AI 只有一层前瞻

下一步引入带剪枝的两层搜索 + 置换表，
或把评估函数换成一个小的价值网络。

UGC 还缺审核与分享

现在只做了上传、配额与删除凭据；
下一步加举报、评分与「一局只用同一套 mod」。

备案完成后切 80 端口 + 域名访问

当前只能用 IP:3000 访问；域名已解析，等 ICP 下来
即可换成 https://yesterday-wagon.xyz。